

NDN IPFS Chain Blog Post — NDP v1.0 Launch

Title: NDN Data Protocol v1.0: One Protocol for Blobs, Models, and Structured Records on IPFS

Slug: ndn-data-protocol-v1-launch

Date: April 18, 2026

Author: Ndibe Kefua — Founder, NDN Analytics

Tags: Protocol, IPFS, Decentralized Storage, AI/ML, Filecoin, Open Source

Excerpt: Today we're publishing the NDN Data Protocol (NDP) v1.0 — a unified specification that brings three database workloads (blobs, AI/ML models, and structured records) under one content-addressed roof on IPFS. Here's why we built it and what it means for production teams.

The Fragmented IPFS Tooling Problem

If you've built anything serious on IPFS, you've probably felt the friction. You need blob pinning — Pinata works fine. But then your application also needs a model registry, so you add HuggingFace. And user records, so you layer in Tableland or a Postgres instance. Now you have three providers, three authentication tokens, three billing meters, and zero unified audit trail.

For a hobby project, this is manageable. For a production team operating in a regulated industry, or serving millions of users, it's a liability.

We felt this problem directly while building the NDN Analytics platform. We needed blobs and AI model registry *and* queryable structured records — all with content addressing, immutable versions, and GDPR-compliant deletion. The existing tooling landscape had pieces but no coherent whole.

So we built the protocol we wished existed.

Introducing the NDN Data Protocol (NDP) v1.0

NDP is a unified, content-addressed data-access protocol for three distinct database workloads:

Blobs DB — arbitrary-byte pinning for files, media, and raw payloads. Fully compatible with the IPFS Pinning Services API v1.0. Every existing IPFS tool works against an NDP provider without modification.

Models DB — an AI/ML model registry with weight shards, model cards, shard maps, and direct-to-GPU streaming. Import any HuggingFace model in one API call. Retrieve individual tensors by their dotted path without downloading the entire checkpoint.

Structured DB — queryable JSON documents with schemas, immutable version chains, collections, and content-addressed views. Write a record, query it with a Mongo-ish filter DSL, and access every historical version by its CID.

Every addressable object in NDP — blob, model version, record, schema, view snapshot — has a canonical Content Identifier (CID). Two NDP implementations producing the same logical content produce the same CID. Cross-provider portability isn't a feature; it's a guarantee born from the protocol design.

How It Works: The Canonical Envelope

The core insight behind NDP is simple: **canonicalization before hashing**.

Every NDP object — record, model version, blob metadata, schema, view snapshot — is serialized as a common JSON envelope. Before hashing, the envelope goes through JCS (JSON Canonicalization Scheme, RFC 8785). JCS guarantees that semantically equivalent JSON objects serialize to byte-identical canonical forms, regardless of key ordering, whitespace, or numeric representation choices made by different implementations.

```
Envelope → JCS (RFC 8785) → SHA-256 → CIDv1 (raw codec)
```

The resulting CID is the object's stable, content-addressed identity. Historical versions are always retrievable. Deletion is provable via crypto-shredding — delete the envelope key on-chain, and every CID addressed by that key becomes permanently unreadable, satisfying GDPR right-to-erasure without repinning a petabyte of data.

Direct-to-GPU Model Streaming

One capability we're particularly excited about is direct-to-GPU streaming for model weights.

The `GET /v1/models/:name/:version/stream` endpoint supports:

- **HTTP Range requests** for random access to any byte offset in any shard
- `?shard=<name>` to stream a specific named weight file
- `?tensor=<path>` (experimental) to stream a single tensor by its dotted path — e.g. `model.layers.0.self_attn.q_proj.weight` — without downloading the entire model
- **Concurrent shard fetching** with bounded parallelism to saturate available bandwidth during training

For teams running inference at scale, this means NDN IPFS Chain can sit between your IPFS-backed model registry and your GPU inference stack without an additional caching layer.

Content-Addressed Views: Structured Data with an IPFS Retrieval Mode

A view is a named, saved query against a collection. At any moment, a view's current result set is canonicalized and CID'd as a view-snapshot envelope. The view's `head_cid` updates on every re-evaluation — but every historical snapshot remains retrievable by its CID.

This gives structured data a native IPFS-style retrieval mode alongside the familiar filter DSL:

- `GET /v1/views/:name` — current view state + head CID
- `GET /v1/views/:cid` — historical view state by CID, even if the view has been re-evaluated hundreds of times since
- `GET /v1/views/:name/stream` — Server-Sent Events stream of CID changes as the view re-evaluates

For audit and compliance teams, this is significant: a view's entire history is content-addressed, versioned, and retrievable. There's no "latest snapshot only" trap.

One Auth, One Tenant, One Billing Meter

The most practical benefit of NDP is also the least glamorous: **one credential, one invoice, one permission model across all three database workloads**.

NDP supports three authentication methods out of the box:

- API keys with fine-grained scopes (blobs:read, models:write, records:admin, etc.)
- JWT (RS256) for browser-side sessions
- SIWE (Sign-In With Ethereum) for dApp developers

Billing is metered per-database — bytes stored and egressed for blobs, weight GB pinned for models, record count and query count for structured data — but invoiced as a single line item.

Open Protocol, Open Implementation

The NDP v1.0 specification is published under **CC-BY 4.0**. The reference implementation is **Apache-2.0**. Any storage provider can implement NDP and become a interoperable node in the network.

We believe the IPFS ecosystem grows when protocols are open and implementations are interoperable. NDN IPFS Chain is one conforming implementation today; we expect others to follow.

The full specification, reference SQL schema, and implementation source code are available in our [public GitHub repository \(https://github.com/dnkefua/ndn-ipfs-chain\)](https://github.com/dnkefua/ndn-ipfs-chain).

What We're Building at NDN Analytics

NDN IPFS Chain is the commercial product built on top of NDP. Our production deployment offers:

- **Multi-region, multi-cloud coverage** — 9 regions across 3 clouds with EU data residency controls
- **Filecoin-backed cold archival** — piece-CID tracking, automatic deal renewal
- **Smart-contract triggers** — pin on Ethereum, Polygon, Arbitrum, Base, Optimism, and Solana events
- **Crypto-shredding** — GDPR-compliant deletion anchored on-chain
- **Enterprise SSO + unified audit log** — daily root CID anchored to an L2 blockchain

We're also an active applicant for the **IPFS Foundation Grant Program**. NDP v1.0 is our headline deliverable; Pinning Services API v1.0 conformance is the prerequisite. Our full grant application is in the repository.

Try It Out

The NDN IPFS Chain dashboard is live at [ndnanalytics.com \(https://ndnanalytics.com\)](https://ndnanalytics.com). Sign up for a free tier — 5 GB, 3 replicas, all features, no credit card required — and explore the NDP API surface.

For the technically curious, the full NDP v1.0 specification is at [specs/ndp_protocol.md \(https://github.com/dnkefua/ndn-ipfs-chain/blob/master/specs/ndp_protocol.md\)](https://github.com/dnkefua/ndn-ipfs-chain/blob/master/specs/ndp_protocol.md) in the repository. It covers the canonical envelope format, CID rules, versioning model, query DSL syntax, API surface, conformance requirements, and security considerations in 9 sections with two appendices.

We welcome feedback, criticism, and collaboration. If you're building something that needs the full production IPFS stack, reach out at [nkefua@ndnanalytics.com \(mailto:nkefua@ndnanalytics.com\)](mailto:nkefua@ndnanalytics.com).

Ndibe Kefua is the founder of NDN Analytics, a company building the data layer for the decentralized web. Based in Nigeria, serving teams globally.